大作业 · 竞赛项目方向

HeavyHeader

C/C++ 构建效能治理平台

用反事实构建实验回答「删掉这条 include 到底能省多少」

课程：智能云原生系统设计与内核生态实践
姓名：王子周 学号：202430552721
完成方式：个人独立完成（1 人）
参赛情况：第四届开放原子大赛 · 2026 开源行业解决方案创新赛 赛道一
2026 年 11 月

代码仓库：https://github.com/wzzrambler/heavyheader
许可证：Apache-2.0 | 单元测试：794 条 | CLI 子命令：15 个

一、项目背景

云原生交付的瓶颈常常不在代码本身，而在构建。一个 C++ 服务的镜像流水线里，编译阶段往往占掉大半时间，而其中绝大部分 CPU 消耗在头文件解析：一个头文件被多个编译单元包含，就要被重复解析多次。这件事在本地开发时是「等一会儿」，在 CI 里就是按分钟计费、按队列排队的真实成本。

问题是，现有工具只回答了一半。ClangBuildAnalyzer、Build Insights（MSVC）、CompileScore 一类工具能告诉你哪个头文件最贵、被包含多少次；include-what-you-use、clang-include-cleaner 一类能告诉你某个 include 在语义上有没有必要。但没有工具回答第二句：把这一行 include 删掉之后，这次构建到底是变快了，还是白改——以及更重要的，删掉之后程序还是不是原来那个程序。

本项目把第二句话做成可执行的东西：对每一条候选 include 做一次反事实构建实验（Counterfactual Build Experiment，CBE）——真的删掉、真的重新编译、真的测时、真的比对汇编指纹，然后给出带证据的结论，并把这个结论固化成 CI 门禁。

问题层次	谁在回答	回答的内容
哪个头最贵	ClangBuildAnalyzer / Build Insights	逐头文件的累计耗时、被包含次数
这条 include 该不该存在	IWYU / clang-include-cleaner	AST 层：符号是否被使用（正确性问题）
删掉它省多少、能不能删	本项目	实测耗时差 + 汇编指纹 + 复测；给出三态结论

表 1 三类问题与对应工具。本项目只做第三行，不与前两类工具争同一件事。

二、技术方案

2.1 一次编译，三个问题

工具不自己实现 C++ 解析器，而是让真实编译器报告自己做了什么：GCC/Clang 用 `-H` 给出包含树，MSVC 用 `/showIncludes`，Clang 可选 `-ftime-trace` 给出逐文件耗时。一次扫描同时回答：① 每个头文件被解析了多少行（成本）；② 每条 include 边的独占闭包（删掉它，只有哪些文件会停止被解析）；③ 这条 include 在文本与符号层是否看起来被使用。

2.2 收益怎么算：归因行数与独占闭包

归因行数 = 头文件自身行数 × 引用它的编译单元数，衡量的是「整个项目为它多解析了多少行」。它是确定性的：同一棵树扫三遍，字节级一致，因此适合做 CI 门禁（不比墙钟时间——后者随负载波动，会误触发）。

独占闭包更接近真实收益：一个头若还能从别的 include 到达，删掉当前这条并不省解析。本项目按「删掉后真正消失的文件集合」计算，并用实测真值验证过这套口径：25 条可删边的假阳性由 45 个

文件降到 0。

2.3 三态决策：不给「疑似」，给结论

候选经探针实测后落进三个互斥结论之一：建议删除（编译通过且耗时下降超过 2% 噪声线）、保留（编译器已否决）、需人工判断（编译可行但收益在噪声内，附倾向与理由）。未实测的候选显示「未实测」——没测过就不下判断，这是本项目最在意的一条纪律。

2.4 证据分档：编译通过不等于程序没变

只测「编译通过 + 耗时下降」是不够的：删掉一条 include 后编译照过、时间还降了，但可能悄悄删掉了一个函数定义，问题要到链接期才暴露。因此判定分三档证据：verified（汇编指纹一致 + 省时过噪声线 + 复测成立）、equivalent（汇编一致但收益不显著）、compiles_only（只能证明编得过，等价性没有证据）。汇编指纹是「代码生成一致」的强证据，不是完整语义等价的证明——这条边界写在报告里，也写在代码注释里。

```
# 一条候选的完整判定链路（摘自 demo 语料实测）
$ heavyheader scan benchmarks/cases/demo --compiler g++ --std c++17
2 TU · 425 条依赖边 · 206,106 行头文件解析量 · 5 个成本候选

$ heavyheader verify benchmarks/cases/demo --ungated-sites --top 5 --runs 3 --confirm 2
## core.h ← util.cpp 状态：compile-error — 'Core' was not declared
## lonely.h ← main.cpp 状态：improved — 基线 720 ms → 移除后 612 ms
证据档：已验证（汇编指纹一致 · 过噪声线 · 复测成立）

# 判据说「在用」的候选，编译器给出了相反结论：判据说不清的地方，编译器说得清。
```

三、架构设计

3.1 分层

工具按「采集层 → 归因层 → 判定层 → 消费层」四层组织，每层只依赖下一层，不做跨层调用。这样做的好处是判定逻辑只有一份实现：CLI、CI 门禁、编辑器扩展读的都是同一份 JSON 结论，不存在「命令行说能删、CI 说不能删」的情况。

```
cli.py 15 个子命令的统一入口（argparse）
├─ drivers/ 编译器驱动：g++ / clang++ / cl（含重试与超时）
├─ scan/ 包含树采集：-H / showIncludes / -ftime-trace 解析
├─ attrib/ 归因层：归因行数、独占闭包、模板实例化、PCH 估算
│  ├─ advice.py 三态决策（纯函数，确定性）
│  ├─ evidence.py 三档证据（verified / equivalent / compiles_only）
│  ├─ crosscheck.py 与 IWYU / include-cleaner 的对位口径（共用一份）
│  └─ diagnosis.py 0 候选时逐条给出被拦原因
├─ probe/ 实测层：verify / pch-verify / forward-verify
└─ report/ Markdown / HTML（自包含单文件）/ JSON
cmake/ meson/ editors/vscode/ 三个接入层，不含第二份判定逻辑
```

3.2 六个刻意的设计选择

选择	理由
零第三方运行时依赖	工具要进别人的 CI。装不上的工具等于不存在；零依赖由 `tests/test_no_deps.py` 守护，新增依赖会让 CI 直接失败
只调编译器，不内嵌 C++ 前端	解析器复现不了真实编译命令，猜错时不报错、只让大量单元以「找不到头文件」失败——报告每个数字都正常，描述的却不是你的项目
门禁只比归因行数，不比墙钟	墙钟随负载波动，会误触发；误触发的门禁会被关掉，关掉的门禁保护不了任何东西
报告是自包含单文件 HTML	评审/同事双击就能看，不需要装服务、不需要联网
失败占比按档措辞	≤20% 说「可能偏低」，>50% 直接说「不足以支撑任何结论」——不让措辞比证据更乐观
实测不可逆	verify 用注释而非删除（.orig 备份 + 退出即恢复），实验必须可回退

3.3 一条命令流水线

15 个子命令不是 15 个卖点，是同一条流水线上的环节：`scan`（发现）→ `verify / pch-verify / forward-verify`（验证）→ `gate / configs / compare`（治理）→ `blame / edge-cost / history / pr-comment`（追溯与呈现）→ `iwyu / include-cleaner / toolchains`（对接与横评）。

四、开源边界与协议

4.1 许可证与理由

项目采用 Apache-2.0。选择理由三条：① 含专利授权条款，对企业 CI 场景不是形式问题；② 与生态中大量 C++ 项目（LLVM 生态、多数基础库）兼容，便于被嵌入；③ 对贡献者与使用者都明确，不需要额外解释。

4.2 运行时边界：零第三方依赖

除 Python 标准库外不引入任何运行时依赖，也不内嵌 C++ 前端。所有「编译相关」的能力都通过调用开发者机器上已有的工具获得：`g++/clang++/cl`（必需）、`gch`（仅 PCH 实测时）、`include-what-you-use` 与 `clang-include-cleaner`（可选对位，缺失时不降级结论、只标「未参与分析」）。零依赖不是口号：`tests/test_no_deps.py` 在 CI 里强制检查。

4.3 与外部工具的边界

外部工具	本项目的用法	边界（不做什么）
GCC / Clang / MSVC	调用其编译与预处理输出，解析包含树与计时	不修改编译器、不注入插件
include-what-you-use	解析其输出，与自己的成本候选对位	本项目构建不出 IWYU 二进制，该对位未做端到端实测，报告正文写明
clang-include-cleaner	同上，且已用本机真实进程输出校准格式	不把「它没建议删」当成「可以删」——编译失败的文件单列且不参与对位
CMake / Meson / VSCode	以插件/构建目标方式接入，调用 CLI	接入层不含第二份判定逻辑

4.4 数据与可复现边界

所有上屏数字都有归档：`benchmarks/results/`（四项目扫描）、`experiments/results/`（逐轮实验，含失败与否定结论）。报告里的每个大数字都附「如何复现」的入口。明确不做的事：不宣称「自动优化」（G2 优化闭环实测为 doctest 1/16、zlib 1/16、json 0/16，如实写在文档里）；不宣称「零假阳性」的绝对版本（该结论只在「删了还能编过」这一层成立）；不把推算当实测（行数与秒之间没有诚实的换算系数）。

4.5 治理与上游协作

仓库含 GOVERNANCE / CONTRIBUTING / CODEOWNERS / SECURITY / Issue 与 PR 模板，以及三个 CI 工作流（3 系统 × 4 Python 版本矩阵、覆盖率、MSVC 真实运行证据）。向上游提交过两个真实优化 PR：doctest#1212（−3.01%）、madler/zlib#1344（−2.125%），截至本文均为 open、未被合并——文中只写「已提交」，不写「已被采用」。

五、验证与实测结果

5.1 四个真实开源项目上的扫描结果

项目	编译单元（成功/失败）	依赖边	头文件解析量	成本候选
fmt	3 / 1	651	331,016	4
nlohmann/json	25 / 0	9,294	4,866,676	24
spdlog	8 / 17	3,246	1,579,902	41
cpp-httplib	18 / 7	14,724	7,523,316	18
合计	54	27,915	14,300,910	87

表 2 失败单元如实计入（缺 CMake 配置头、缺 OpenSSL 头等），没有为凑数字跳过。

5.2 三个关键收益数字

① 单条最大收益：zstd 1.5.6 上删一条 include（`pool.c → allocations.h`），编译耗时 384 ms → 205 ms（−46.6%），解析文件数 184 → 32。② 完整构建对照：nlohmann/json 63 个编译单元，墙上时间 −7.1%、编译 CPU −11.4%，4,264,414 条断言逐个一致。③ 与竞品同任务对照：clang-include-cleaner 在同一批候选上给出 0 条删除建议、101 条补充建议；本工具给出的一条经三维度实测有效（−43.5%）。

同一行 include 在两种构建配置下答案可能相反：zstd 的这条在默认配置下删了能编过且汇编一致，加上 `-DZSTD_MULTITHREAD` 之后删了直接编译失败。「该不该删」不是源码的属性，是构建配置的属性——这条结论现在是一条命令（`configs`）。

5.3 如实记录的边界

边界	实际情况
自动优化闭环	成立但有限：doctest 1/16、zlib 1/16、json 0/16；不宣传「自动优化」
候选判定精度	222 个候选里 187 个编译不过（文本层的原理性局限，AST 层才能根治）
前向声明建议	三个真实语料上候选数为 0（合成语料 301.7→41.9 ms 只证明机制，不当效能证据）
PCH 实测	json 24 个单元里 9 个加 PCH 后编不过——采用 PCH 的真实代价，估算表达不了
平台覆盖	MSVC 只跑通 scan 与 gate；增量构建税仍在原型阶段，不作结论引用

六、代码与复现入口

仓库：`https://github.com/wzzrambler/heavyheader`（Apache-2.0，含全部源码、测试、基准数据与实验归档）。已在 PyPI 发布 v0.1.0，可直接安装。随本作业一并提交源码压缩包（`HeavyHeader-source.zip`：src/ tests/ benchmarks/ experiments/ docs/ 与工程配置，不含构建产物与演示视频）。

```
# 安装与最小验证
pip install heavyheader
heavyheader scan benchmarks/cases/demo --compiler g++ --std c++17
heavyheader verify benchmarks/cases/demo --ungated-sites --top 5 --runs 3 --confirm 2
heavyheader gate benchmarks/cases/demo --baseline baseline.json

# 本地开发与回归
python -m pytest --no-header # 794 passed / 10 skipped
ruff check src tests # 0 违规
```

测试共 794 条，覆盖解析器（用真实编译器输出逐字复制的样本）、归因算法、门禁、实测流程、并发与失败路径；10 条因环境缺失（meson/ninja 或特定编译器）跳过，而不是假装通过。

七、个人分工与组内评分

本作业个人独立完成（1 人，未组队）。按课程要求说明分工：从选题、资料检索、方案设计、代码实现、测试、实测实验、文档撰写到参赛材料制作，全部由本人完成；外部资源仅使用公开文档与开源项目源码，未使用他人未公开材料。

分工项	内容	占比
选题与调研	构建成本工具调研（ClangBuildAnalyzer / IWYU / include-cleaner 等）、确定「反事实」这一差异点	10%
方案与实现	四层架构、15 个子命令、编译器驱动与解析器、三态决策与三档证据、门禁与追溯	45%
实验与验证	四个真实语料扫描、单条与完整构建对照、竞品同任务对照、失败与否定结论归档	25%
测试与工程化	794 条单元测试、零依赖守护、CMake/Meson/VSCode 接入、CI 三工作流	12%
文档与参赛材料	作品介绍文档（35 页）、选题报告、演示视频、本大作业文档	8%

表 3 个人分工（1 人，全部工作由本人承担）。

组长姓名	主要职责	组内评分
王子周（组长，1 人组）	全部：选题、方案、实现、测试、实验、文档、参赛材料	—（个人作业）

表 4 按课程要求的分工表。个人作业无组内互评，故评分栏不填。

八、心得体会

差分测试的思路可以迁移。课程里用「同一程序在两个内核上跑、对比输出」定位内核语义差异；本项目做的是同一件事的内核外版本：同一条 include 在两份代码上跑（删与不删），对比编译器输出。区别是这里没有「标准答案」，所以必须自己造真值——汇编指纹、解析文件数、断言计数，都是为了让「有没有变」有确定性答案，而不是靠感觉。

证据链比数字好看更重要。项目里最花时间的不是让工具算出收益，而是让每个数字都能被追到出处：为什么是 −46.6% 而不是别的、这条候选为什么判「保留」、这个门禁为什么可信。写进文档的每个数字都附复现入口，不利数据也留在正文而不是附注里。

否认自己是很容易的事。项目最初的说法是「帮你找到可验证的优化」，实测结果把这个说法推翻了（16 条候选只有 1 条真有效）。后来改成「把『看起来能省』变成『确认能省 / 确认不能省』」——口径变窄了，但每一句都能被验证。这是这门课之外我最想记住的一条。

九、参考资料

1. ClangBuildAnalyzer：https://github.com/aras-p/ClangBuildAnalyzer
2. include-what-you-use：https://github.com/include-what-you-use/include-what-you-use
3. Clang include-cleaner：https://clang.llvm.org/extra/clang-include-cleaner/
4. Aras Pranckevičius，The Blowup Factor（头文件成本度量口径来源）
5. Bitsquid，Header Hero（首个体量化头文件成本的工具）
6. GCC 手册：-H / -ftime-report；Clang 手册：-ftime-trace；MSVC：/showIncludes
7. 本项目仓库内文档：docs/benchmark-report.md（全部实测）、docs/methodology.md（指标定义与局限）、docs/innovation.md（创新点与不声称清单）、docs/competitor-matrix.md（竞品逐项对照）

> 本文所有数字均可在仓库中复现：`benchmarks/results/` 与 `experiments/results/` 保存了每一轮的原始记录。